

Windows 64ビット Electronと C言語 32bit DLLの共存

ffi_honji v1.0.0

個人事業よいけ 石丸 裕
yutaka_ishimaru@yahoo.co.jp



開発の動機

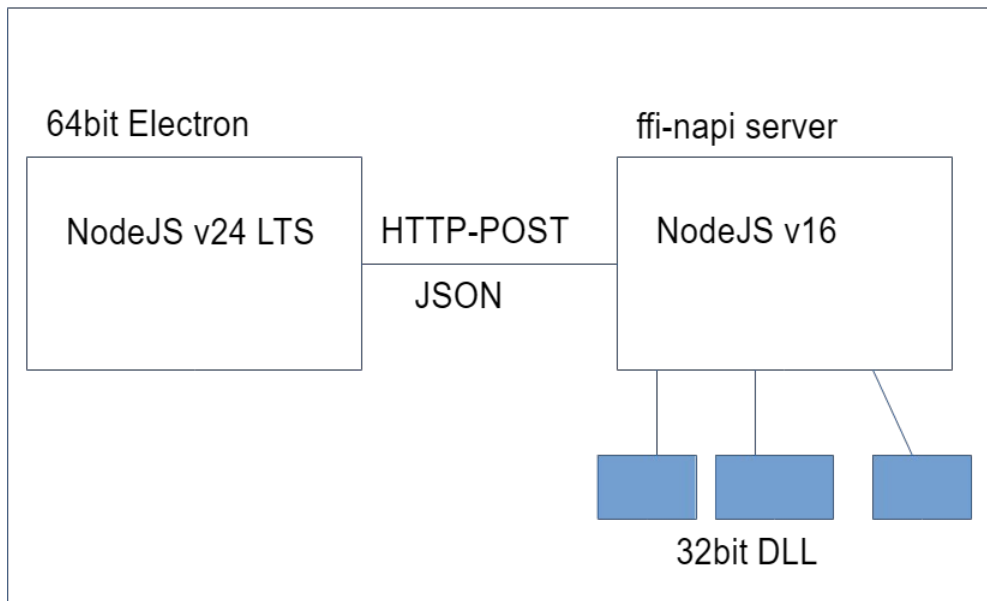
-
- AIがElectronは64bit版LTSで開発したほうがいいと言ってくる
- 一方、業務の現場では32bit版DLLは安定していて捨てがたい

→64bitと32bitを両立したい



システム構成

localhost



問題点

- ffi-napi単体の場合はパススルーでいけそう。
- ref-array-napi, ref-struct-napiを使うケース。つまり、OUTPUT引数をとる場合が、プロトコルをJSONにする都合上つまづいた。



妥協策

- オブジェクトの配列で構造体のIN/OUT引数や配列のIN/OUT引数を指定するようにした。
 - 構造体IN/OUTの場合。[{a:"char", b:"short", c:"int"}, <配列長>]
 - 配列IN/OUTの場合。 ["int32", <配列長>]
- ・ ref-struct-napi, ref-array-napiを駆使しなければならない場合は、サーバープログラムを独自拡張で対応する。



DLL関数例 1

- Cの関数その 1 (device1_32.dll)
__declspec(dllexport) int sm2cin() {
- int xrtcd1;
- xrtcd1 = 2;
- xrtcd1 = xrtcd1 + 1;
- return xrtcd1;
- }



呼出例 1

- 従来のffi-napiの呼出法をほぼ同じように利用可能。
-
- ```
let adll1 = await ffi_honji_cli("./dll/device1_32.dll",
```
- ```
    { "sm2cin": ["int",[]] });
```
- ```
const val = await adll1.ffi_call("sm2cin");
```
- ```
console.log( "sm2cin()=", val);    // 3
```



DLL関数例 2

- Cの関数その 2 (device2_32.dll)
- 第 3 引数が値を返す OUTPUT 引数になっている
- ```
__declspec(dllexport) int sm2cin(int a, int b, int* sum) {
 *sum = a + b;

 return 0;
}
```





## 呼出例 2

- `sm2cin`関数の第3引数の型宣言を独自拡張した。
- `let adll2 = await ffi_honji_cli( "./dll/device2_32.dll",`
- `{ "sm2cin": ["int",["int", "int", ["int",1]]] });`
- `const val = await adll2.ffi_call("sm2cin", 1, 2, [0]);`
- `console.log( "sm2cin=",val); // 0`
- `console.log( "outArg[2]=",adll2.mojson.xotarg[2]); // 3`



# 型の拡張 1

- 前項の例で、DLL関数の第2引数がOUTPUT引数の `int *` 型であることをどうやってJSON表現するか？
- 本プログラムでの解決法：
- `int`型の長さ1の配列を用意し、先頭ポインタをDLL関数の第2引数に渡した。
- 型の表現では `[“int”, 1]` と表現した。



# DLL関数例 3

- Cの関数その3 (device3\_32.dll)
- 関数の第 1 引数が構造体にINPUT引数で参照渡し。
- typedef struct { char a; short b; int c; } TSTRUC;
- \_\_declspec(dllexport) int sm2cin( TSTRUC \*s, int \*sum) {
- \*sum = s->a + s->b + s->c;
- return 0;
- }



## 呼出例 3

- `sm2cin`関数の第 1 引数の型に構造体を表現するよう拡張。
- `let adll3 = await ffi_honji_cli( "./dll/device3_32.dll",`
- `{ "sm2cin": ["int",[[{"a":"char", "b":"short", "c":"int"},1], ["int",1]]]`
- `});`
- `const val = await adll3.ffi_call("sm2cin",[{a:1, b: 2, c: 3}], [0]);`
- `console.log("sm2cin()=", val); // 0`
- `console.log("outArg[1]=",adll3.mojson.xotarg[2]); // 6`



## 型の拡張 2

- 前項の例で、DLL関数の第 1 引数が構造体のINPUT引数で、
- `struct { char a; short b; int c };` 型をどうやって宣言するか？
- 本プログラムでの解決法：
- 長さ 1 の構造体配列を用意し、構造体の先頭ポインタをDLL関数の第 1 引数に渡した。
- 型の表現では`[{a:"char", b:"short", c:"int"},1]` とした。
- 呼出時の値表現では、第 1 引数を、`[{a:1, b:2, c:3}]` とした。



# まとめ

- ffi-napiのありがちな呼出例の 7 割は、本関数のインターフェースでラッパー可能。
- ref-struct-napi, ref-array-napiが複雑に交錯する場合は、ffiサーバー側にプロトコルを拡張して、ffi-napi呼出を実装することで対応するのがよさそうです。

